

Testing Mini-Spark

Este documento describe la estrategia completa de testing para el framework Mini-Spark, incluyendo tests unitarios, de integración y end-to-end (E2E).

Archivos de Scripts Disponibles

Tests Unitarios

- **runTests/runUnitTests.sh** - Script para Linux/Mac (Bash)
- **runTests/runUnitTests.bat** - Script para Windows (CMD/PowerShell)
- **Tiempo:** ~4-5 segundos | **Tests:** 76 | **Dependencias:** Solo Go

Tests de Integración

- **runTests/runIntegrationTests.sh** - Script para Linux/Mac (Bash)
- **runTests/runIntegrationTests.bat** - Script para Windows (CMD/PowerShell)
- **Tiempo:** ~40-45 segundos | **Tests:** 4 | **Dependencias:** Cluster activo

Tests End-to-End (E2E)

- **runTests/runE2ETests.sh** - Script para Linux/Mac (Bash)
- **runTests/runE2ETests.bat** - Script para Windows (CMD/PowerShell)
- **Tiempo:** ~2-5 minutos | **Tests:** 5 | **Dependencias:** Cluster Docker

Guía de Uso

Ejecución Simple

Solo hacer doble clic (Windows) o ejecutar en terminal:

```
# Tests que NO requieren cluster:  
./runTests/runUnitTests.sh           # runTests/runUnitTests.bat (Windows)  
  
# Tests que SÍ requieren cluster (ejecutar `make up` primero):  
./runTests/runIntegrationTests.sh    # runTests/runIntegrationTests.bat  
(Windows)  
./runTests/runE2ETests.sh           # runTests/runE2ETests.bat (Windows)
```

Preparación del Entorno

```
# 1. Iniciar cluster (para tests de integración y E2E)  
make up  
  
# 2. Verificar que esté funcionando  
curl http://localhost:8080/health
```

```
# 3. Ejecutar cualquier test
./runTests/runE2ETests.sh # Ejemplo
```

Interpretación de Resultados

- **Verde/PASS:** Todo funcionando
- **Rojo/FAIL:** Revisar logs y troubleshooting
- **Timeout:** Problemas de recursos o red

Ejecución de Tests

Para Tests Unitarios

```
# Linux/Mac
./runTests/runUnitTests.sh

# Windows
.\runTests\runUnitTests.bat
```

Para Tests de Integración

```
# Prerequisito: Iniciar cluster (con docker)
make up

# Linux/Mac
./runTests/runIntegrationTests.sh

# Windows
.\runTests\runIntegrationTests.bat
```

Tests End-to-End

```
# Prerequisito: Cluster con workers (con docker)
make up

# Linux/Mac
./runTests/runE2ETests.sh

# Windows
.\runTests\runE2ETests.bat
```

Ejecutar Todos los Tests

```
# Linux/Mac - Secuencial
./runTests/runUnitTests.sh && ./runTests/runIntegrationTests.sh &&
./runTests/runE2ETests.sh

# Windows - Secuencial
.\runTests\runUnitTests.bat && .\runTests\runIntegrationTests.bat &&
.\runTests\runE2ETests.bat

# Manual con go test (cualquier OS)
go test ./... -v                                # Solo unitarios
go test ./tests/ -run TestIntegration -v         # Solo integración
go test ./tests/ -run TestE2E -v                # Solo E2E
```

Cobertura y Especificación de Tests

Tests Unitarios (76 tests - 100% passing)

Los test unitarios se encuentran en los siguientes archivos, los cuales validan las siguientes funcionalidades:

common/types_test.go (29 tests)

- Estructuras de datos básicas (Job, Task, DAG, Node, Edge)
- Validación de configuraciones de jobs
- Serialización/deserialización JSON
- Validación de estados (JobStatus, TaskStatus)

common/cache_test.go (15 tests)

- Sistema de cache en memoria con LRU
- Spill-to-disk automático cuando se excede capacidad
- Operaciones thread-safe y concurrencia
- Limpieza automática de archivos temporales

master/scheduler_test.go (15 tests)

- Algoritmo de planificación DAG (topological sort de Kahn)
- Asignación de tareas a workers disponibles
- Manejo de dependencias entre nodos
- Tolerancia a fallos y reasignación de tareas

worker/executor_test.go (17 tests)

- Ejecución de operadores: read_csv, map, filter, flat_map, reduce_by_key, aggregate, join
- Particionamiento hash-based de datos
- Manejo de archivos CSV y operaciones de transformación
- Persistencia de resultados y cleanup automático

Tests de Integración (4 tests - 100% passing)

Los tests de integración validan la comunicación entre componentes en el cluster:

TestIntegrationWordCount

- **Propósito:** Valida pipeline básico de procesamiento de texto
- **Validación:** Conteo correcto de palabras específicas en archivo text.csv

TestIntegrationAggregation

- **Propósito:** Valida operaciones de agregación numerica
- **Validación:** Suma correcta de precios en archivo sales.csv

TestIntegrationPipeline

- **Propósito:** Valida pipeline multi-etapa con join
- **Validación:** Join correcto y agregación de datos relacionales

TestIntegrationFailureRecovery

- **Propósito:** Valida tolerancia a fallos básica
- **Validación:** Job completa exitosamente a pesar de posibles fallos

Tests End-to-End (5 tests - 100% passing)

Los tests E2E validan el funcionamiento completo del sistema distribuido con múltiples workers:

TestE2EWordCountMultiNode

- **Propósito:** Valida distribución básica de procesamiento de texto
- **Características:** 6 particiones distribuidas entre 3 workers
- **Validación:** Resultados distribuidos coherentes entre particiones
- **Por qué este test:** Representa el caso de uso más común (MapReduce básico)

TestE2EFailureRecoveryMultiNode

- **Propósito:** Valida tolerancia a fallos en entorno distribuido
- **Características:** Mismo job que WordCount con validación de recuperación
- **Validación:** Sistema se recupera automáticamente de fallos de workers
- **Por qué elegimos este test:** Tolerancia a fallos es crítica en sistemas distribuidos

TestE2EComplexPipelineMultiNode

- **Propósito:** Valida pipeline multi-etapa con filter + aggregate
- **Validación:** Pipeline complejo ejecutado correctamente en cluster
- **Por qué elegimos este test:** Simula casos de uso reales de analítica de datos

TestE2EConcurrentJobs

- **Propósito:** Valida ejecución simultánea de múltiples jobs

- **Características:** 3 jobs diferentes ejecutándose concurrentemente
- **Validación:** Todos los jobs completan sin interferencia mutua
- **Por qué elegimos este test:** Sistemas reales manejan múltiples cargas de trabajo

TestE2ELoadBalance

- **Propósito:** Valida distribución eficiente de carga entre workers
- **Características:** 6 particiones distribuidas automáticamente
- **Validación:** Carga distribuida equitativamente entre workers disponibles
- **Por qué elegimos este test:** El balanceo es una característica crucial en performance de clusters

Métricas de Testing

Tipo de Test	Cantidad	Tiempo Ejecución	Coverage
Unitarios	76	~4-5 segundos	Lógica completa
Integración	4	~38 segundos	Comunicación master-worker
E2E	5	~3 minutos	Sistema completo distribuido
TOTAL	85	~140 segundos	100% funcionalidad

Criterios de Selección de Tests E2E

Los tests E2E fueron elegidos para cubrir los escenarios más críticos de un sistema distribuido:

1. **Funcionalidad básica** (WordCount): Patrón MapReduce fundamental
2. **Tolerancia a fallos:** Recuperación ante fallos
3. **Pipelines complejos:** Casos de uso reales de procesamiento de datos
4. **Concurrencia:** Capacidad de manejar múltiples trabajos simultáneamente
5. **Balanceo de carga:** Distribución eficiente de trabajo entre workers

Solución de Problemas

Errores Comunes y Soluciones

Problema	Síntoma	Solución
Go no encontrado	<code>command not found: go</code>	Instalar Go y agregarlo al PATH
Cluster no disponible	<code>connection refused</code> <code>localhost:8080</code>	Ejecutar <code>make up</code>
Tests lentos	Timeouts frecuentes	Aumentar recursos Docker, cerrar programas
Archivos faltantes	<code>file not found: text.csv</code>	Verificar archivos en <code>app/data/</code>

Problema	Síntoma	Solución
Workers inactivos	Solo 0-1 workers en cluster	Esperar 30s después de <code>make up</code> o reiniciar el cluster

Comandos de Diagnóstico

```
# Verificar instalación Go
go version

# Verificar cluster activo
curl http://localhost:8080/health

# Ver logs de Docker
docker-compose logs

# Reiniciar cluster
make down && make up

# Test manual simple
go test ./common -v

#Veriificar archivos
# Para ver todos los archivos de resultados
ls -la app/results/

# Para buscar archivos específicos de tests de integración
find app/results/ -name "*integration*" -type f

# Para verificar archivos de datos de entrada
ls -la app/data/

# Para verificar archivos específicos como pipeline results
find app/results/ -name "*pipeline*" -type f
```

Resumen Ejecutivo

Métrica	Valor	Estado
Tests Totales	85	✓ 100% Pasando
Cobertura Funcional	Completa	✓ Todos los componentes
Tiempo Total	~5-7 minutos	✓ Automatizado
Plataformas	Windows + Linux/Mac	✓ Cross-platform

Capacidades Validadas

- **Lógica de negocio** (76 tests unitarios)
- **Comunicación distribuida** (4 tests integración)

- **Sistema completo** (5 tests E2E)
- **Tolerancia a fallos** y recuperación automática
- **Balanceo de carga** y distribución eficiente
- **Pipelines complejos** multi-etapa
- **Concurrencia** y jobs simultáneos

Tips para Usuarios Técnicos

Para Desarrolladores

```
# Desarrollo diario - Solo unitarios (5 segundos)
./runUnitTests.sh

# Pre-commit - Validación completa (4 minutos)
make up
./runIntegrationTests.sh && ./runE2ETests.sh
```

Para QA/Testing

```
# Suite completa con reporte
./runUnitTests.sh > unit_results.log
./runIntegrationTests.sh > integration_results.log
./runE2ETests.sh > e2e_results.log
```